

Space-Efficient and Output-Sensitive Algorithm

for the Longest Common Bitonic Subsequence (LCBS)



American International University- Bangladesh

ALGORITHM

Section: H

DIRECTED BY:

DR.MD.MANZURUL HASAN

PRESENTED BY:

SAYEDA TANJILA AHMED

APURBA

ID: 24-56599-1



- Space-Efficient
- Output-Sensitive
 - Longest
 - Common
 - Bitonic
- Subsequence

What is LCBS?

Example:

Sequence: 1-

$A = \{5, 6, 7, 9, 4, 3\}$

Sequence: 2-

$B = \{5, 6, 4, 3, 2, 1\}$

$LCBS = \{5, 6, 4, 3\}$





Hypothesis:

Can LCBS be computed with less than quadratic time, less than quadratic space, or both, on practically relevant instance families?

Research gap:

Section:2-

- Large DP tables
- Consumes high memory
- High time complexity
- LIS : $O(n \log n)$ time
- LBS: $O(n^2)$ time and space

Quadratic-time dynamic-programming

Linear-space refinement

$\Theta(nm)$ time using $\Theta(\min\{n,m\})$ additional memory



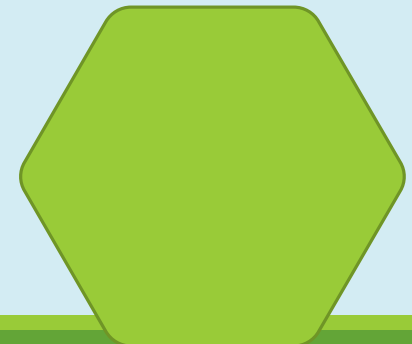
Methodology:

■ **Table 1** Summary of our LCBS algorithms. Here σ is the alphabet size and $\mathcal{M}$ the number of value-equal matches (i, j) ; $\ell = \text{LCBS}(A, B)$.

Algorithm	Time complexity	Extra space
Baseline DP, rolling rows (Sect. 2)	$O(nm)$	$O(m)$
Output-sensitive sparse DAG (Sect. 3)	$O((n + m) \log n + \mathcal{M} \log^2 \mathcal{M})$	$O(\mathcal{M})$

Section:3-

- Output-sensitive sparse-DAG formulation.
- Correspondence between longest paths and LCBS.
- Algorithm that adapts number of matching pairs.



A Linear-Space $\Theta(nm)$ -Time Algorithm for LCBS

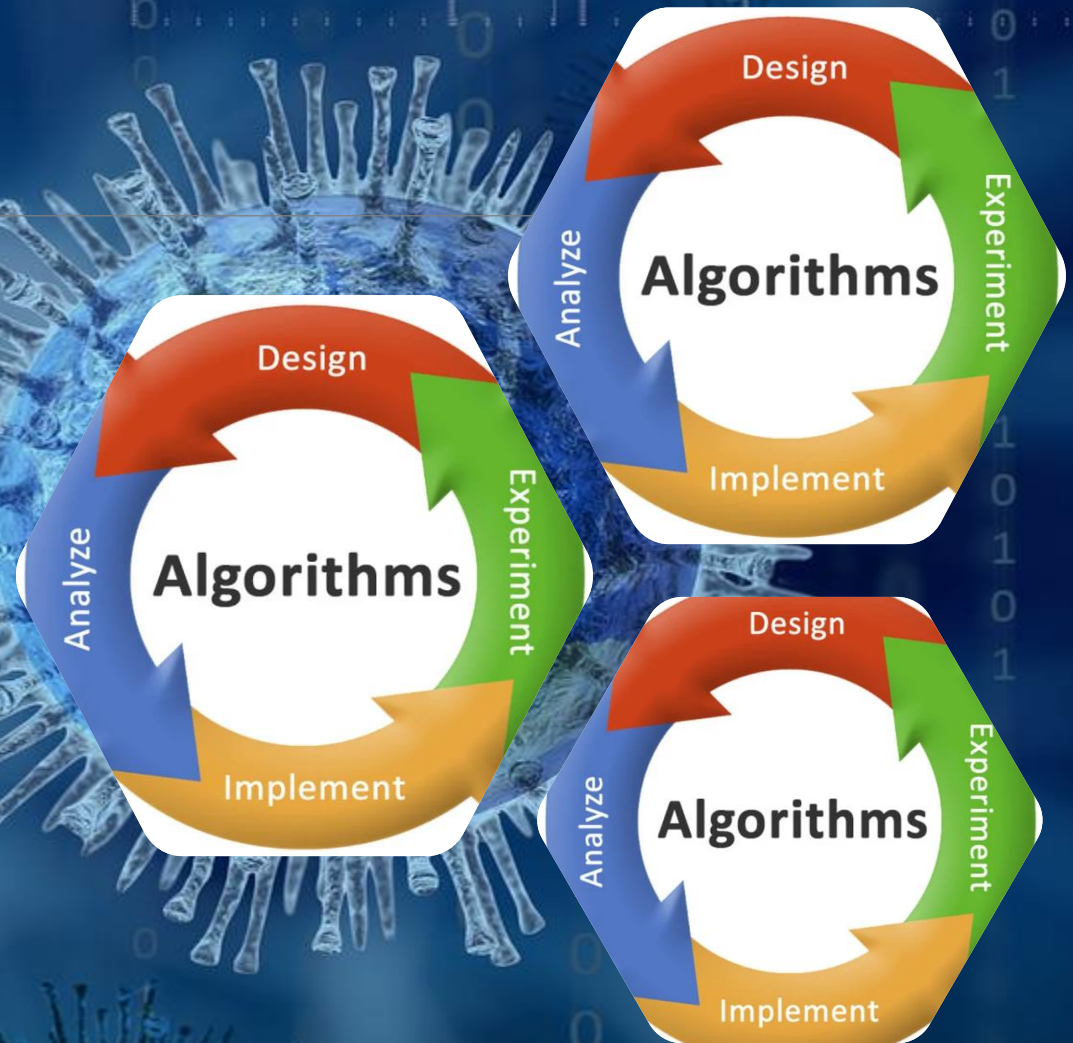
$A[0\dots n-1]$ and $B[0\dots m-1]$ in $\Theta(nm)$ time and space.

An Output-Sensitive Sparse-DAG Algorithm for LCBS

$$M = |\{(i,j) \mid A[i] = B[j]\}|$$

$O(n+m) \log n + C$ time and $O(M)$ space

- Forward Scan
- Backward Scan



DISCUSSION AND CONCLUSION

- **Linear-space DP**
- **Sparse-DAG**



The results show that the space-efficient algorithm reduces memory from $\Theta(nm)$ to $\Theta(\min\{n,m\})$, while the output-sensitive algorithm runs faster than quadratic time when the number of matching pairs is small.





Algorithm

- Worst-case still quadratic
- Hard to implement
- Rolling array info doesn't loss
- Not always faster

POSSIBLE LIMITATIONS

References

- 1 Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. Tight hardness results for LCS and other sequence similarity measures. In 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, pages 59–78. IEEE, 2015.
- 2 Anadi Agrawal and Paweł Gawrychowski. A faster subquadratic algorithm for the longest common increasing subsequence problem. In 31st International Symposium on Algorithms and Computation (ISAAC 2020), pages 4–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.
- 3 Karl Bringman and Marvin Künnemann. Multivariate fine-grained complexity of longest common subsequence. In Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 1216–1235. SIAM, 2018.
- 4 LechDuraj. A sub-quadratic algorithm for the longest common increasing subsequence problem. In 37th International Symposium on Theoretical Aspects of Computer Science (STACS 2020), pages 41–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.
- 5 LechDuraj, Marvin Künnemann, and Adam Polak. Tight conditional lower bounds for longest common increasing subsequence. *Algorithmica*, 81:3968–3992, 2019.
- 6 Michael L Fredman. On computing the length of longest increasing subsequences. *Discrete Mathematics*, 11(1):29–35, 1975.
- 7 Daniel S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975.
- 8 James W. Hunt and Thomas G. Szymanski. A fast algorithm for computing longest common subsequences. *Commun. ACM*, 20(5):350–353, May 1977. doi:10.1145/359581.359603.
- 9 MdTanzeem Rahat, Md Manzurul Hasan, and Debajyoti Mondal. A space-efficient algorithm for longest common almost increasing subsequence of two sequences. In International Workshop on Combinatorial Algorithms, pages 228–241. Springer, 2025.
- 10 Gde B Robinson. On the representations of the symmetric group. *American Journal of Mathematics*, pages 745–760, 1938.
- 11 Craige Schensted. Longest increasing and decreasing subsequences. *Canadian Journal of mathematics*, 13:179–191, 1961.
- 12 I-Hsuan Yang, Chien-Pin Huang, and Kun-Mao Chao. A fast algorithm for computing a longest common increasing subsequence. *Information Processing Letters*, 93(5):249–253, 2005.



Space-Efficient and Output-Sensitive Algorithms for the Longest Common Bitonic Subsequence

1. Abstract:

This paper studies the Longest Common Bitonic Subsequence (LCBS) problem and presents space-efficient and output-sensitive algorithms that significantly reduce memory usage and runtime for large inputs.

3. Example & Motivation

An example illustrates how LCBS combines increasing and decreasing parts around a peak and why it is useful for analyzing biological signals, financial trends, and time-series data.

5. Quadratic Dynamic Programming Approach:

This section presents a classic $\Theta(nm)$ dynamic programming solution that computes LCBS by combining increasing and decreasing common subsequences.

6. Complexity Analysis

While worst-case time remains quadratic, memory is reduced to linear space, and runtime improves significantly when the number of matching pairs is small.

9. Discussion :

The study shows that LCBS computation becomes practical when memory usage is reduced and unnecessary comparisons are avoided. The sparse-DAG approach demonstrates that exploiting matching structure significantly improves performance on real-world datasets.

Conclusion :

The paper successfully provides the first space-efficient and output-sensitive exact algorithms for the LCBS problem. These methods bridge the gap between theoretical hardness and practical applicability of LCBS.

10. Future Scope :

Future work may explore faster data structures or parallel implementations to further reduce runtime.



Longest Common Subsequence

2. Introduction :

LCBS captures real-world rise-fall patterns in two sequences, but existing solutions are too slow or memory-heavy for large data.

4. Methodology:

The authors first optimize the classic dynamic programming approach to use linear space, then propose a sparse-DAG formulation LCBS efficiently by operating only on matching elements.

7. Output-Sensitive Sparse-DAG Algorithm (Section 3):

By modeling only matching pairs as a sparse DAG, the algorithm runs faster when matches are few, making performance depend on output size.

8. Results:

The space-efficient algorithm reduces memory from $\Theta(nm)$ to $\Theta(\min\{n, m\})$, while the output-sensitive algorithm runs faster than quadratic time when the number of matching pairs is small.

References:

- 1 Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. Tight hardness results for LCS and other sequence similarity measures. In 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, pages 59–78. IEEE, 2015.
- 2 Anadi Agrawal and Paweł Gawrychowski. A faster subquadratic algorithm for the longest common increasing subsequence problem. In 31st International Symposium on Algorithms and Computation (ISAAC 2020), pages 4–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.
- 3 Karl Bringman and Marvin Künnemann. Multivariate fine-grained complexity of longest common subsequence. In Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 1216–1235. SIAM, 2018.
- 4 LechDuraj. A sub-quadratic algorithm for the longest common increasing subsequence problem. In 37th International Symposium on Theoretical Aspects of Computer Science (STACS 2020), pages 41–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.
- 5 LechDuraj, Marvin Künnemann, and Adam Polak. Tight conditional lower bounds for longest common increasing subsequence. Algorithmica, 81:3968–3992, 2019.
- 6 Michael L Fredman. On computing the length of longest increasing subsequences. Discrete Mathematics, 11(1):29–35, 1975.
- 7 Daniel S. Hirschberg. A linear space algorithm for computing maximal common subsequences. Communications of the ACM, 18(6):341–343, 1975.
- 8 James W. Hunt and Thomas G. Szymanski. A fast algorithm for computing longest common subsequences. Commun. ACM, 20(5):350–353, May 1977. doi:10.1145/359581.359603.
- 9 MdTanzeem Rahat, Md Manzurul Hasan, and Debajyoti Mondal. A space-efficient algorithm for longest common almost increasing subsequence of two sequences. In International Workshop on Combinatorial Algorithms, pages 228–241. Springer, 2025.
- 10 Gde B Robinson. On the representations of the symmetric group. American Journal of Mathematics, pages 745–760, 1938.
- 11 Craige Schensted. Longest increasing and decreasing subsequences. Canadian Journal of mathematics, 13:179–191, 1961.
- 12 I-Hsuan Yang, Chien-Pin Huang, and Kun-Mao Chao. A fast algorithm for computing a longest common increasing subsequence. Information Processing Letters, 93(5):249–253, 2005.



POSTER
REPRESENTATION



American International University-Bangladesh

ALGORITHM
Section: H

ON BEHALF OF:

SAYEDA TANJILA AHMED APURBA

ID: 24-56599-1

Email: apurbasayedatanjilaahmed@gmail.com

THANK YOU